

momo

商業資料分析統整課程

Capstone for Business Analytics

Group 1 朱柏諺 羅名邑 賴可婕 黃伊華 葉光筠



組員分工



朱柏諺 B10704031

專案管理、環境架設、強化
學習設計



羅名邑 B10704017

袋箱分類、推薦箱子、篩選
箱子



葉光筠 B09207008

推薦袋子、袋子輔助貼標



賴可婕 B09703038

單多箱分類、模型設計



黃伊華 B10102111

單多箱分類、推薦袋子

Agenda

專案
摘要

執行
內容

未來
應用

- 1 專案目標與簡介
- 2 裝箱判斷流程
- 3 單多箱分類
- 4 袋箱分類
- 5 推薦最佳袋子
- 6 推薦最佳箱子
- 7 企業實務應用

提出新的模型架構，為每筆訂單選擇合適的包材，提升裝載率並優化裝箱流程

專案目標

透過創新方法架構，優化紙箱與袋子選擇流程，提升裝載率並減少包裝材使用

執行方法

單多箱分類

利用規則和模型判斷訂單是否需要多箱，再將訂單拆為單箱銜接後續流程

袋箱分類器

用模型判斷訂單是否適合袋裝，減少後續計算量

篩選可能袋箱

以規則排除不適合的袋箱，減少模型計算負擔

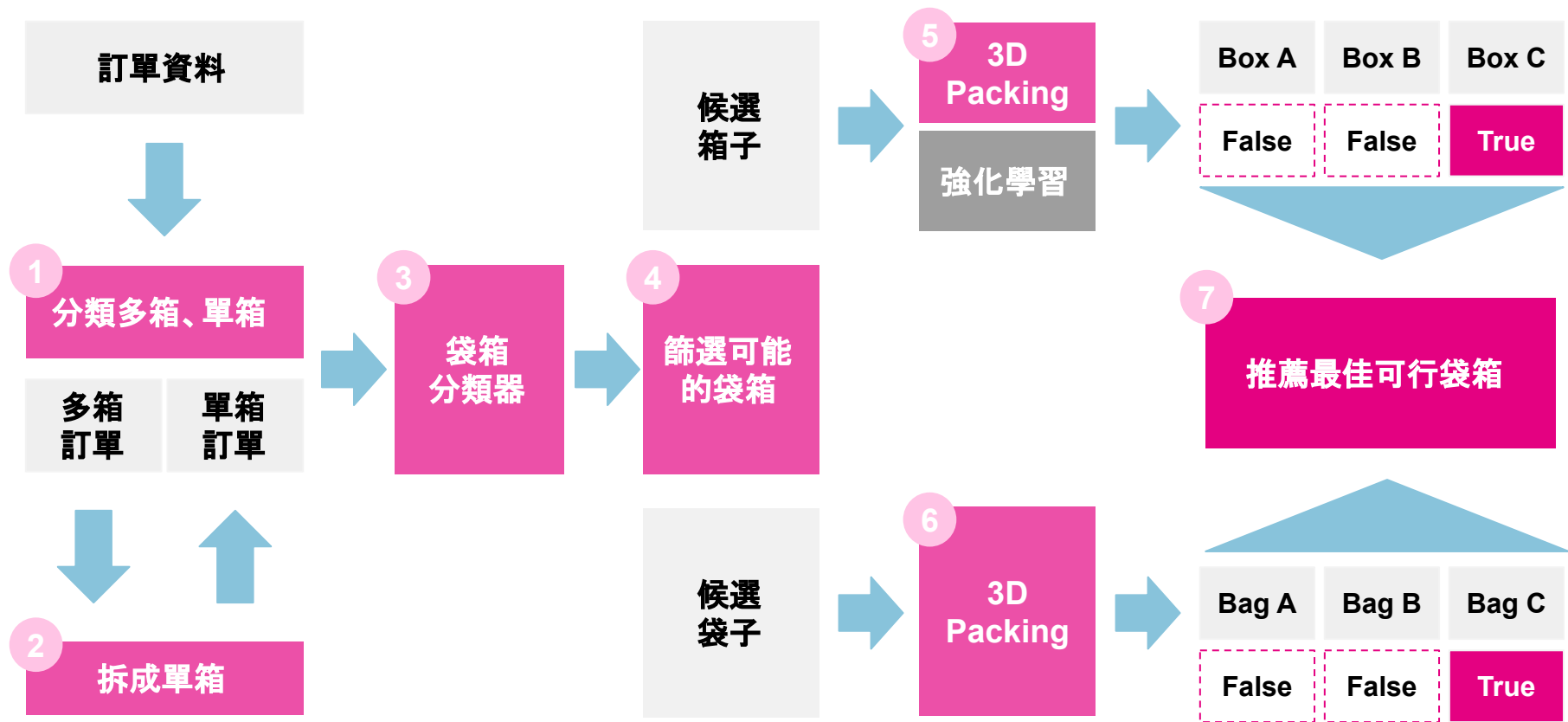
模型判斷

使用 3D Packing 模擬計算裝箱可行性，並推薦最適袋箱

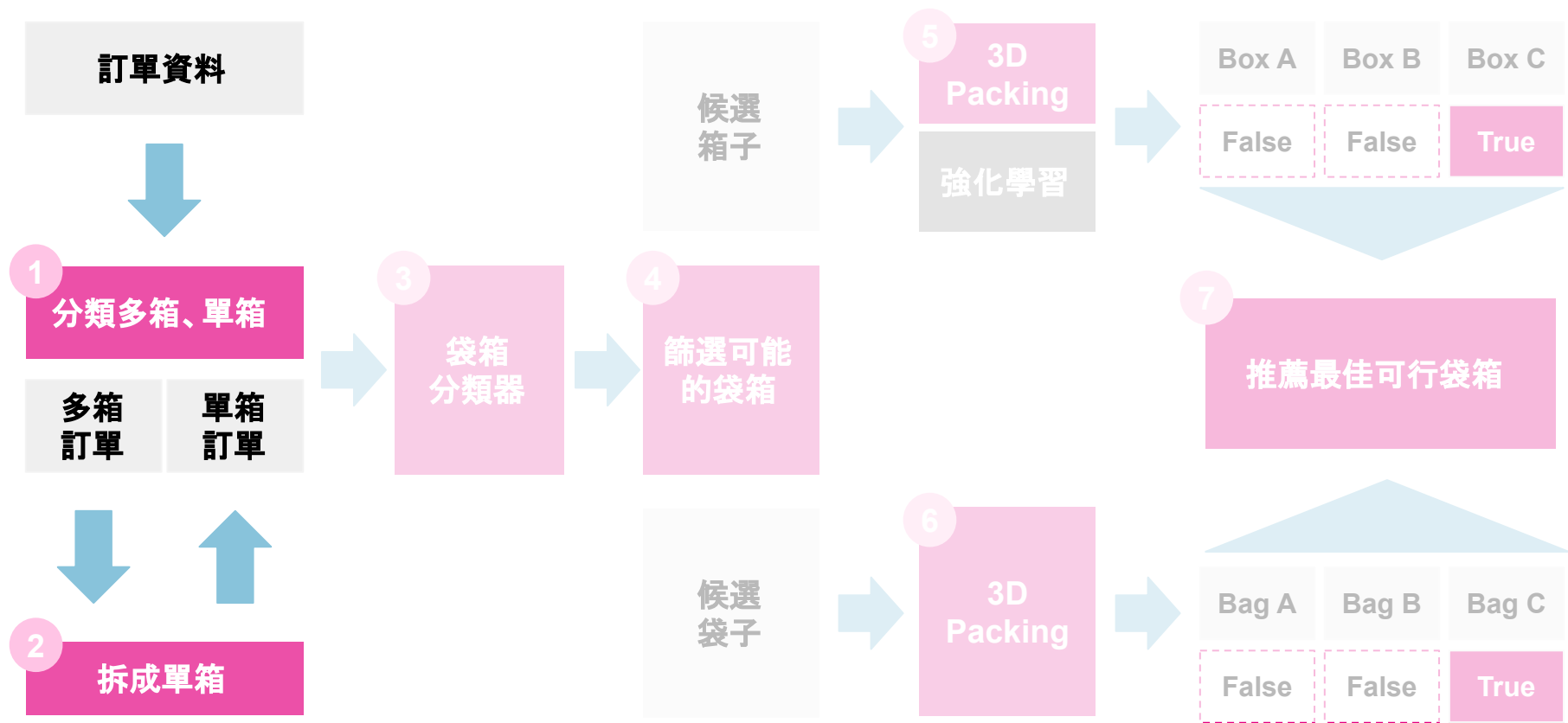
輸出

針對每筆訂單，給予一個 **推薦最佳箱子** 與一個 **推薦最佳袋子**（若有），供裝台人員選擇

裝箱判斷流程



裝箱判斷流程 | 單多箱分類



目標與特徵 | 分析 EDA 結果以了解訂單資料特徵，並以此設計二階段分箱流程

訂單拆箱 分裝比例

- 96.7% 為單箱訂單，表示多數訂單只需要一個箱子
- 3.3% 為多箱訂單，須考量重量、體積等特殊規則



單多箱 分類目標

將訂單統一拆成單箱處理，提升一致性

避免體積過大或商品太多導致裝箱錯誤

提升裝箱效率，減少無效模擬次數

為後續選箱邏輯提供準確前置判斷

二階段分箱 流程圖

讀取
訂單資料



單多箱分類

單箱
訂單
多箱
訂單



多箱分群

單多箱分類 | 先將訂單分為單箱與多箱訂單，再透過分類模型拆分多箱訂單

單多箱分類

Step1. Rule-Based 判斷

根據體積、重量、宅配方式的不同，初步判斷訂單是否應該為單箱、多箱或進一步交由模型判斷

- 訂單只有一項商品 → 判定單箱(0) → 共 **9,776 筆** 訂單
- 若體積或重量超過上限 → 判定多箱(1) → 共 **3 筆** 訂單

Step2. 分類模型補充判斷

對於 Step1 判斷為「不確定」的訂單，使用後續訓練好的分類模型進行分類 → 共 **29,300 筆** 訂單

混淆矩陣

	實際單箱(0)	實際多箱(1)
預測單箱(0)	39,036	8
預測多箱(1)	4	31

評估指標

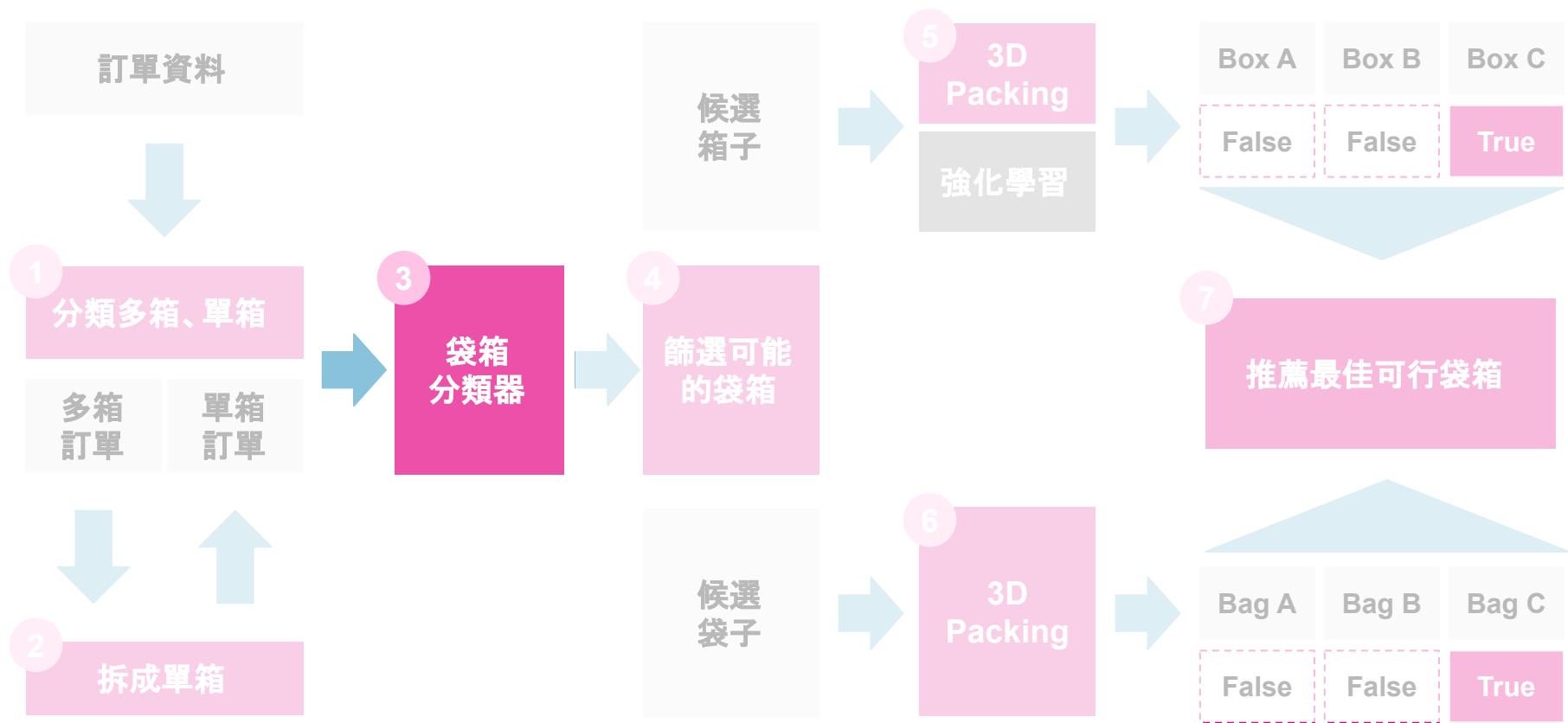
	Precision	Recall	F1 score	Support
單箱(0)	99.99%	99.99%	99.99%	39,040
多箱(1)	88.57%	79.49%	83.78%	39

多箱分群

- **多箱商品分群**：設定給分標準，將「確認為多箱的訂單」依據商品特性進行分群。
- **給分標準**：包含運費基礎分、同商品優先、體積平衡、重量平衡、易碎品隔離、配送方式加權

指標	數值
總比對訂單數	39
分箱完全一致的訂單數(箱數商品皆一致)	21
分箱不一致的訂單數	18
分箱一致率	53.85%

裝箱判斷流程 | 袋箱分類器

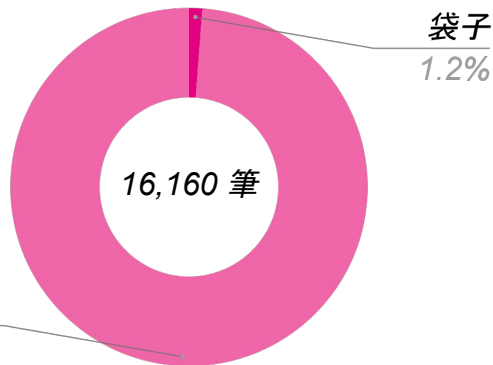


袋箱分類 | 利用商品特徵，讓分類模型判斷「該筆訂單是否可能可以使用袋子」

“ 目標：將所有訂單中，有可能可以使用袋子的訂單篩選出來，以利後續最佳袋箱判斷 ”



實際使用之袋箱比例



將以下商品特徵餵入模型判斷

體積系列

商品總體積/ 最大商品體積/ 平均商品體積/ 體積第三四分位數/ 體積標準差

重量系列

總重量/ 最大商品重量/ 平均商品重量/ 商品密度(總重量/ 總體積)

個數系列

總商品數量/ 相異商品個數/ 易碎品個數/ 易碎品佔比

配送方式

宅配/ 超商/ i 郵箱

商品種類

大分類/ 中分類

袋箱分類 | 模型篩選掉 76% 的訂單不需進入 3D Packing 判斷, 大幅提高效率

方法

模型透過該筆訂單的商品特徵, 判斷「是否為可能使用袋子的訂單」

評估標準

模型 **Recall 高於 9 成** (能找出大部分使用袋子的訂單) 的狀況下, Precision 要越高越好 (預測的越準確)



模型選擇 & 結果評估

使用模型

最佳 (Precision, Recall)

最終結果

羅吉斯迴歸	(0.029, 0.909)
Random Forest	(0.031, 0.909)
Balanced Random Forest	(0.047, 0.909)
LightGBM	(0.032, 0.909)
XGBoost	(0.033, 1.000)
SVM	(0.028, 0.909)

- Cross Validation
- Fine Tune

0

Precision

Recall

F1-score

Support

0.998

0.775

0.873

756

1

0.101

0.950

0.182

20

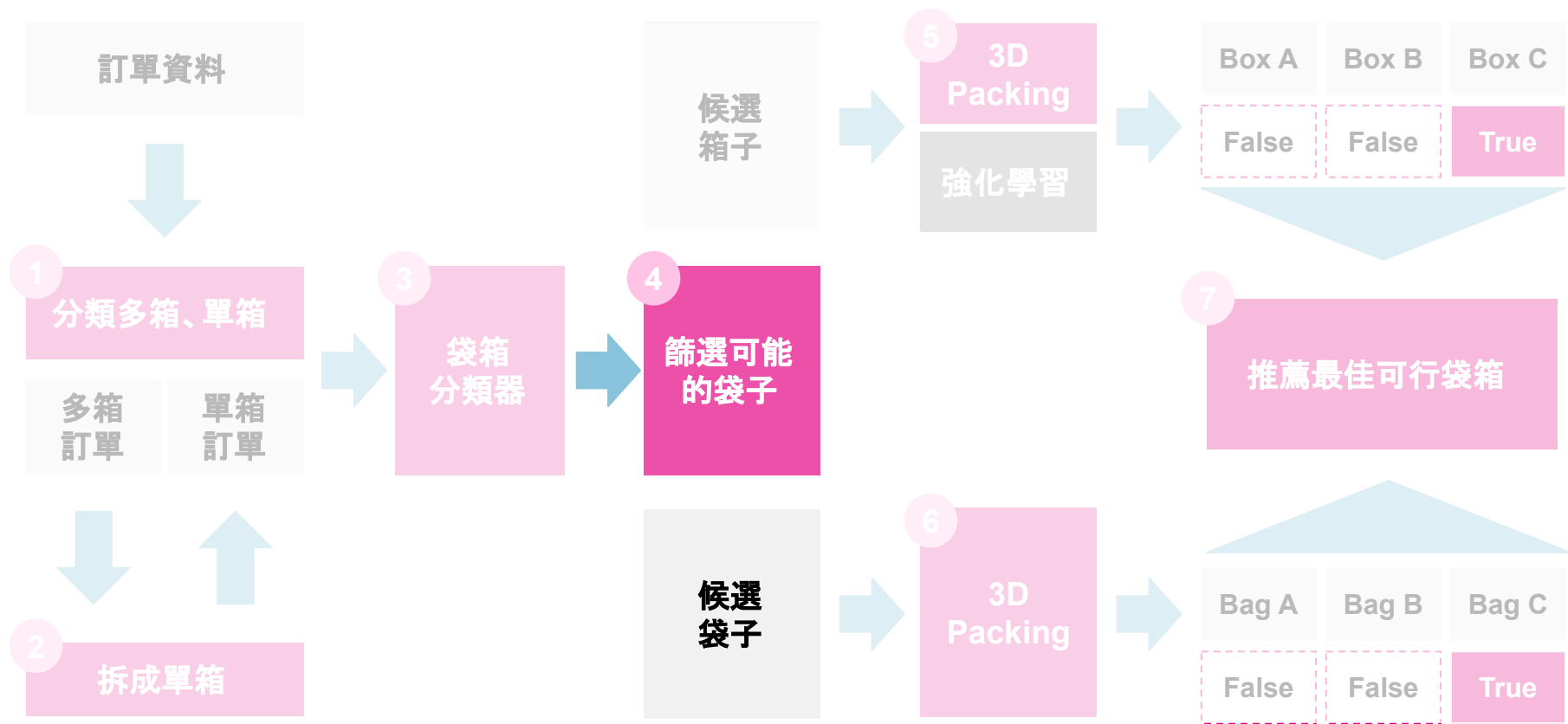
Accuracy

0.780

776

“節省 76% 的訂單不需要判斷袋子”

裝箱判斷流程 | 篩選可能的袋子



篩選可能袋子 | 透過五項規則，將後續進入模型之袋子從 455 個減少至 21.41 個

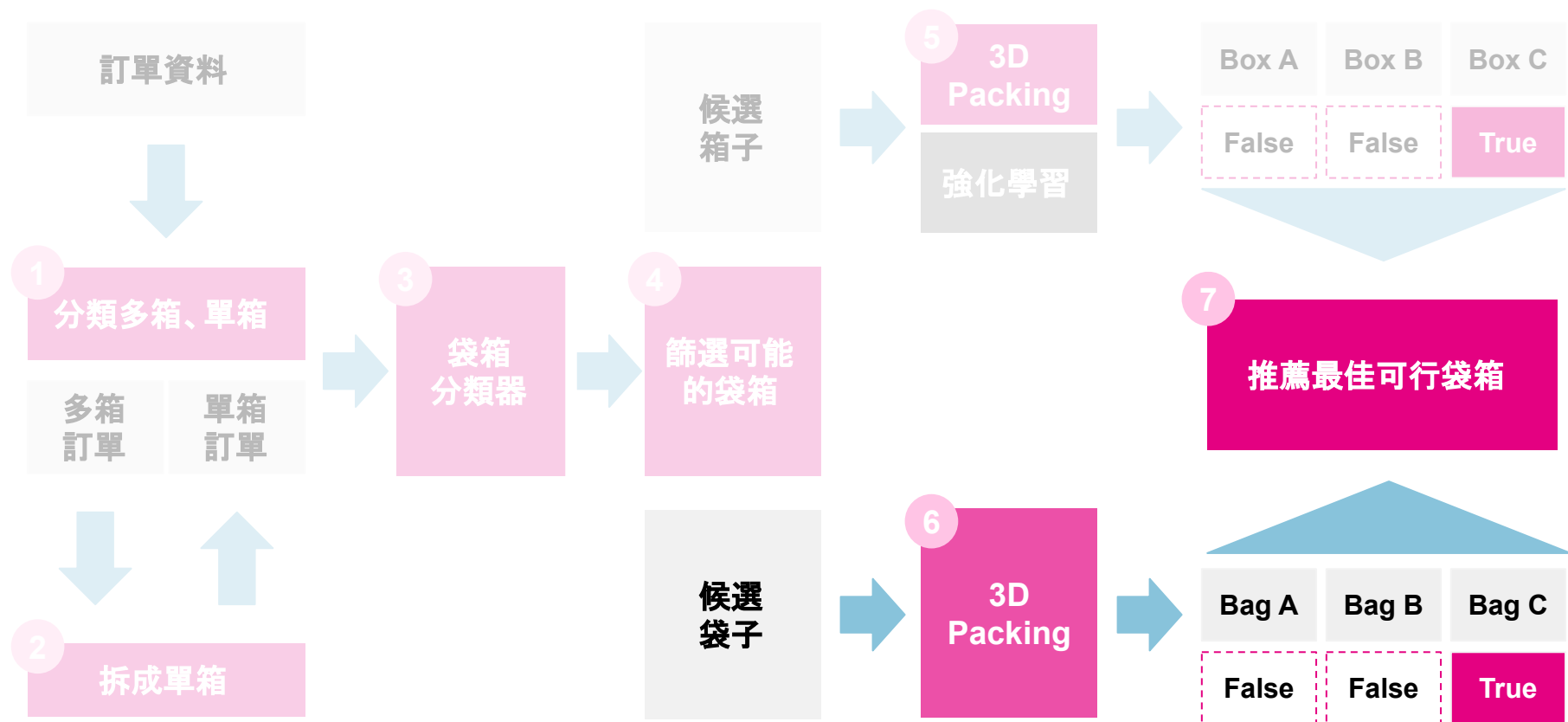
“ 目標：透過硬性規則結合專案目標，篩選掉「該筆訂單不可能使用的袋子」，加速後續模型判斷效率 ”

篩選規則	1	無法裝入袋子	- 若商品列在「附錄不可用 BOPO 商品(未附)」中，則不能使用破壞袋 - 若「商品總體積 > 破壞袋的最大體積」，則不能使用破壞袋
	2	易碎品有無	- 若含有易碎品，則刪除 PO 系列袋子，只能使用 BO 系列 - 若不含易碎品，則刪除 BO 系列袋子，只能使用 PO 系列
	3	篩掉體積過小袋子	- 若「袋子體積 * 最佳安全邊際 < 該筆訂單商品總體積」，則從清單中剔除 - 考量到商品長寬高相乘，不一定為實際商品的體積，透過安全邊際放大袋子來因應
	4	篩掉體積過大袋子	- 若「商品體積 * 最大安全邊際 < 袋子體積」，則從可用清單中剔除 - 此步驟僅刪除與商品體積差距過大的袋子，因此透過安全邊際放大商品來因應
	5	篩掉尺寸過小袋子	- 若「其中一個商品的最長邊 > 袋子最長邊」，則從可用清單中剔除 - 考量到商品最長邊可能可以透過折疊等方式縮短，因此透過安全邊際放大袋子來因應



結果	篩選前	篩選後
	455 個袋子 / 每筆訂單	平均 21.41 個袋子 / 每筆訂單

裝箱判斷流程 | 3D Packing 推薦最佳可行袋箱



推薦最佳袋子 | 利用 3D Packing, 找到能裝入且編號最小的袋子作為最佳解

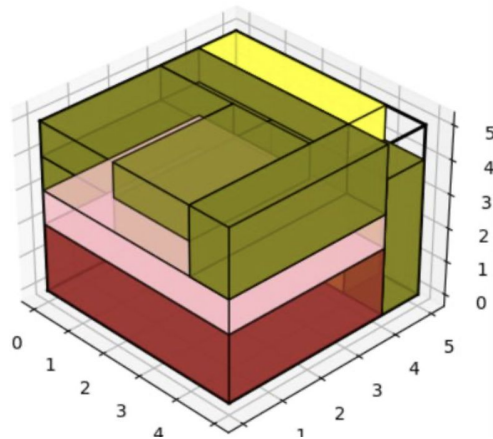
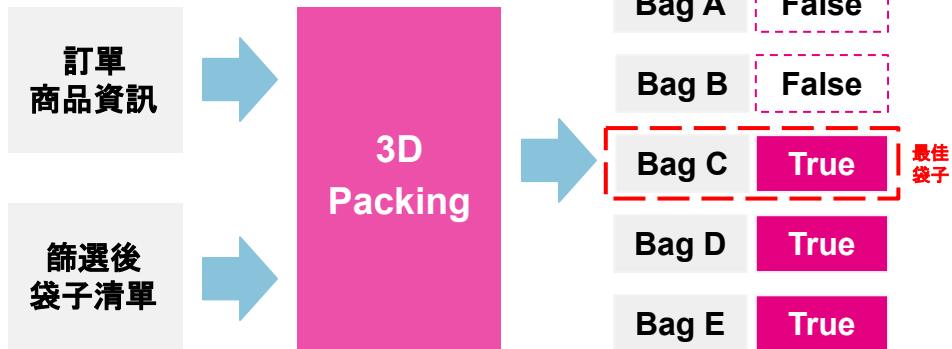
“ 目標: 依據每筆訂單的商品資料, 找到「能裝入 & 編號最小的」袋子 ”

使用 套件

3D-bin-packing

透過商品與袋子的尺寸資料, 能模擬裝箱、了解「這些商品是否能完全裝入這個袋子」

判斷 流程



3D-bin-packing 裝箱示意圖

推薦最佳袋子 | 利用準確率評估模型表現, 整體準確率最高可達 82%

定義

準確率

對 True label 為「箱子」的訂單:

- 沒有任何一個袋子變形擺得下的訂單數 / 總訂單數

對 True label 為「袋子」的訂單:

- 一般版本: 預測為 True Label 的訂單數 / 總訂單數
- 寬容版本: 預測為 True Label 或比 True Label 大一號的訂單數 / 總訂單數



整體準確率最高 (Most Accurate)

袋箱準確率均高 (Most Balanced)

模型評估

準確率

模型意義

全部訂單

不推薦
袋子

推薦袋子

一般版

寬容版

82.01%

88.24%

10.53%

26.32%

多數 True Label 為箱子訂單, 不會額外推薦袋子, 可減少裝台人員選擇要用箱子或袋子的時間

全部訂單

不推薦
袋子

推薦袋子

一般版

寬容版

65.08%

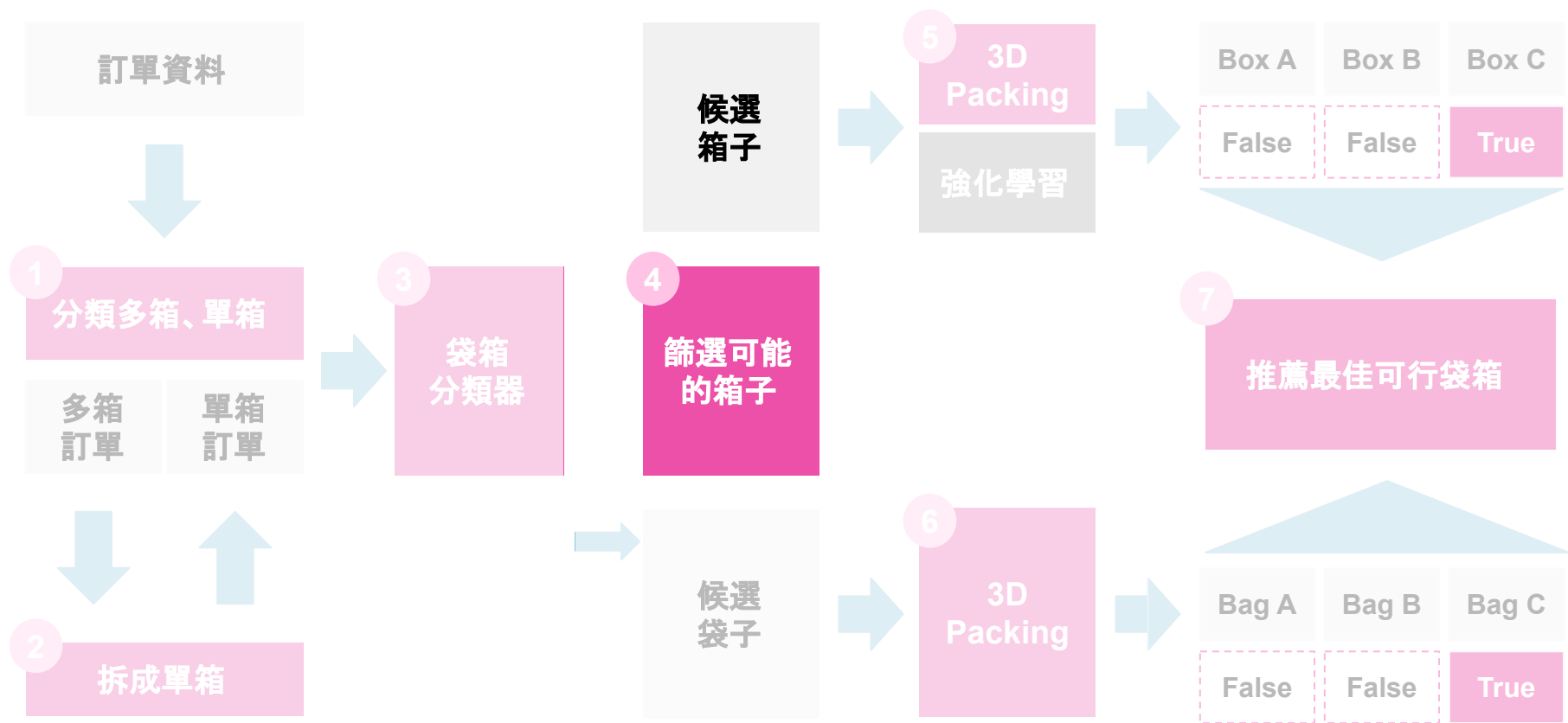
66.47%

26.32%

52.63%

部分箱子訂單有潛力改用袋子, 增加出貨時整體袋子使用率, 更符合公司的環保政策

裝箱判斷流程 | 篩選可能箱子



篩選可能箱子 | 透過五項規則，將後續進入模型之箱子從 25 個減少至 9.89 個

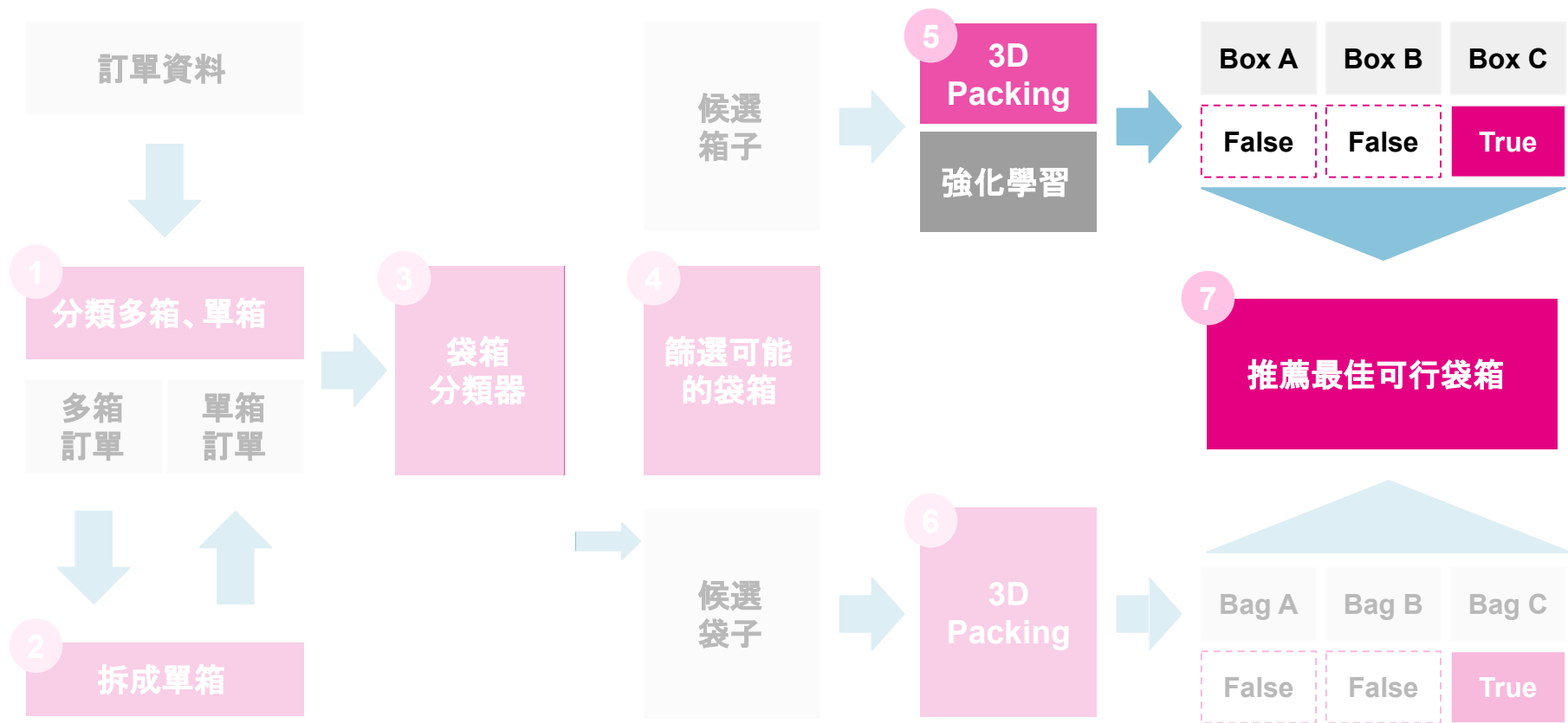
“ 目標：透過硬性規則結合專案目標，篩選掉「該筆訂單不可能使用的箱子」，加速後續模型判斷效率 ”

篩選規則	1 配送方式	- 三種配送方式(宅配、超商、郵箱)各有其尺寸與重量限制 - 針對該訂單之配送方式，僅挑選符合條件之箱子
	2 篩掉體積過小箱子	- 若「箱子體積 * 最小安全邊際 < 該筆訂單商品總體積」，則從清單中剔除 - 考量到商品長寬高相乘，不一定為實際商品的體積，透過安全邊際放大箱子來因應
	3 篩掉體積過大箱子	- 若「商品體積 * 最大安全邊際 < 箱子體積」，則從可用清單中剔除 - 此步驟僅刪除與商品體積差距過大的箱子，因此透過安全邊際放大商品來因應
	4 篩掉尺寸過小箱子	- 若「其中一個商品的最長邊 > 箱子最長邊」，則從可用清單中剔除 - 考量到商品最長邊可能可以透過折疊等方式縮短，因此透過安全邊際放大箱子來因應
	特 新增可用箱子	- 若「篩選後已經沒有可用箱子」，則從可用清單中加入最小的 5 個箱子 - 考量到商品過小導致所有箱子都過大的情況，因此透過加入最小的箱子來因應



結果	篩選前	篩選後
	25 個箱子 / 每筆訂單	平均 9.89 個箱子 / 每筆訂單

裝箱判斷流程 | 3D Packing



實作過的方法: 強化學習、3D Packing 套件、Tree-Based Classifier

	Pros	Cons	訓練表現
強化學習	可以學習複雜策略 根據資料學習	策略難以收斂	差
3D Packing 套件	方便觀測裝箱邏輯 有解釋性	觀測值、Reward、動作 都需要反覆調整嘗試(實作困難)	較佳
Tree-Based	根據資料學習 實作簡單	缺乏真實裝箱邏輯	較佳

推薦最佳箱子 | 利用 3D Packing, 找到能裝入且編號最小的箱子作為最佳解

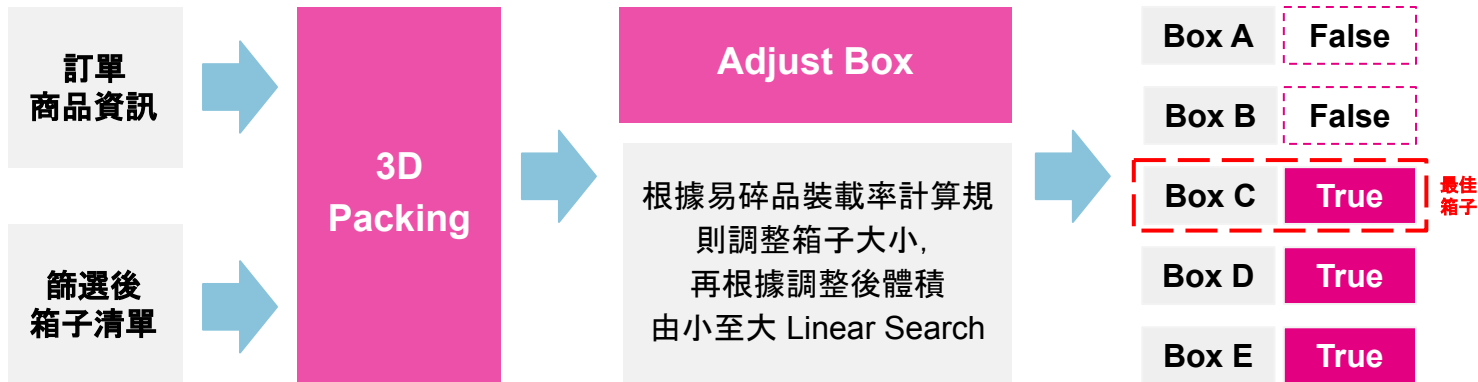
“ 目標: 依據每筆訂單的商品資料, 找到「能裝入 & 編號最小的」箱子 ”

使用 套件

3D-bin-packing

透過商品與袋子的尺寸資料, 能模擬裝箱、了解「這些商品是否能完全裝入這個箱子」

判斷 流程

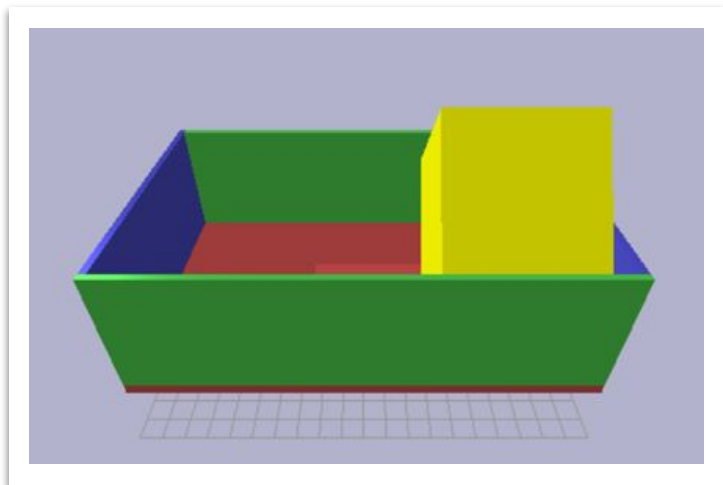


計算 Model 準確度 | 根據模型預測的行為與模擬結果，評估所選箱子是否正確



指標	定義	結果
Accuracy	模型預測準確度	48.75%
Utilization	裝載率，易碎品依規定調整計算	59.43%
Filter Box Error	經過規則篩選，True Label 不在 Filter List 裡面	5.78% (618)
Overestimate Error	高估裝載率錯誤：選擇的箱子比 True label 小	24.68% (2,488)
Underestimate Error	低估裝載率錯誤：選擇的箱子比 True label 大	26.56% (2,677)

PyBullet Packing Environment | 根據企業資料建立物理引擎模擬環境



Application

訓練強化學習

測試 Rule-Based 裝箱演算法

Functions

支援三軸自由旋轉

模擬重力、穩定度、摩擦力

容易客製化調整(如易碎品調整箱子大小)

實作 Multiprocessing, 提升模擬效率

企業實務應用 | 裝載率提升的企業應用與效益：從包貨流程優化到 ESG 實踐

方法	效益	企業實務應用
單多箱分類及多箱分群邏輯	目前系統只推薦單箱，造成多箱訂單裝箱困難。提出的單多箱分類與分群架構事先規劃多箱分裝，能避免實務上降低效率與包材浪費	提供明確裝箱 SOP 提高工作效率
推薦箱子訂單潛在的可用袋子	目前整體袋子使用率非常低，但實際上袋子比起箱子更環保、經濟，現在 Most balanced 推薦袋子模型會把部分原本使用 BOX 的訂單轉為推薦使用 BAG	提升袋子使用率 支持企業的環保政策
提供客製化環境優化情境模擬	建構具備旋轉、擺放順序與商品擺放順序的模擬環境，往後可以部署演算法同時貼近真實情境模擬，也可以用於訓練強化學習環境	優化包材利用 降低包材費與運費
		增強顧客體驗與滿意度



感謝聆聽！



朱柏諺
專案負責人



羅名邑
專案組員



葉光筠
專案組員



黃伊華
專案組員



賴可婕
專案組員

QA

- 強化學習為何無法收斂
-

單箱多箱判斷 | 重量與體積依運送方式不同之限制

	宅配	i郵箱	超商
重量限制	20 公斤		10 公斤
體積限制	無限制	總長(長+寬+高) < 105 公分，單邊長 < 45 公分	
體積對應最大箱子	長:56.1 公分 寬:35.1 公分 高:40.8 公分	長:44.1 公分 寬:29.1 公分 高:28.8 公分	

多箱分類 | Ruled-Based 程式邏輯 & 分箱流程

數據處理

將訂單資料中的長、寬、高、重量、數量轉為標準數值格式。

限制參數設定

根據配送方式(超商、i郵箱、宅配)設定最大體積、重量上限。

運費計算模組

根據不同配送規則套用分級運費。

分箱流程

- 以商品體積由大至小排序。
- 每放置一個商品時，根據「運費級距」、「同商品集中」、「體積 / 重量平衡」、「易碎品隔離」、「配送類型」計算打分(Score)。
- 根據最低分數選擇最適合的箱子。
- 若無適合箱子，則開新箱。

保底規則

因已經經過isMultiBox分為多箱的訂單才會進入這個步驟，因此強制切成兩箱以上。

輸出結果

每箱清單、分箱記錄、預估運費報表。

多箱分類 | 設定 Score 打分邏輯

目標設定

- 讓商品盡可能合理、成本低、出貨安全地放進現有箱子
- 原則：分數越低 → 越適合放進該箱子



Score 計算公式

項目	說明	權重影響
運費基礎分 (Fee × 100)	根據這個箱子的「預期運費」設定基礎分數。運費越高，起始分數越高。	最主要影響來源
同商品優先 (Same Item Bonus)	如果同一箱內已經有相同商品，則減少分數（鼓勵同商品放一起）。	-20分 (Bonus)
體積平衡 (Volume Balance)	新商品放入後，箱子體積越接近平均理想體積，分數越低。	平衡加分 (10×偏差量)
重量平衡 (Weight Balance)	新商品放入後，箱子重量越接近平均理想重量，分數越低。	平衡加分 (40×偏差量)
易碎品隔離 (Fragile Penalty)	如果易碎品與非易碎品混裝，則加分（懲罰）。	+20分 (Penalty)
配送方式加權 (Delivery Adjustment)	超商、郵箱配送時，重量平衡再額外強化（增加0.5×重量偏差分）。	特殊調整

$$\begin{aligned}
 \text{Score} = & (\text{預估運費} \times 100) \\
 & + (\text{同商品放一起 Bonus}) \\
 & + (\text{體積平衡懲罰}) \\
 & + (\text{重量平衡懲罰}) \\
 & + (\text{易碎品混裝懲罰}) \\
 & + (\text{超商/i郵箱時額外重量懲罰})
 \end{aligned}$$

推薦最佳袋子之成效與準確率

	整體準確率最高 (most accurate)				袋箱準確率均高 (most balanced)			
	全部訂單	不推薦袋子	推薦袋子		全部訂單	不推薦袋子	推薦袋子	
			一般版	寬容版			一般版	寬容版
準確率	82.01%	88.24%	10.53%	26.32%	65.08%	66.47%	26.32%	52.63%
訂單數	189	170	19	19	189	170	19	19
正確推薦比數	155	150	5	2	123	113	10	5
高估裝載率 錯誤數	20	20	0	0	57	57	0	0
低估裝載率 錯誤數	14	0	14	17	9	0	9	14
模型意義	對大多數 True Label 為箱子的訂單不會額外推薦袋子, 可減少裝台人員選擇要用箱子或袋子的時間				部分箱子訂單有潛力改用袋子, 增加出貨時整體袋子使用率, 更符合公司的環保政策			

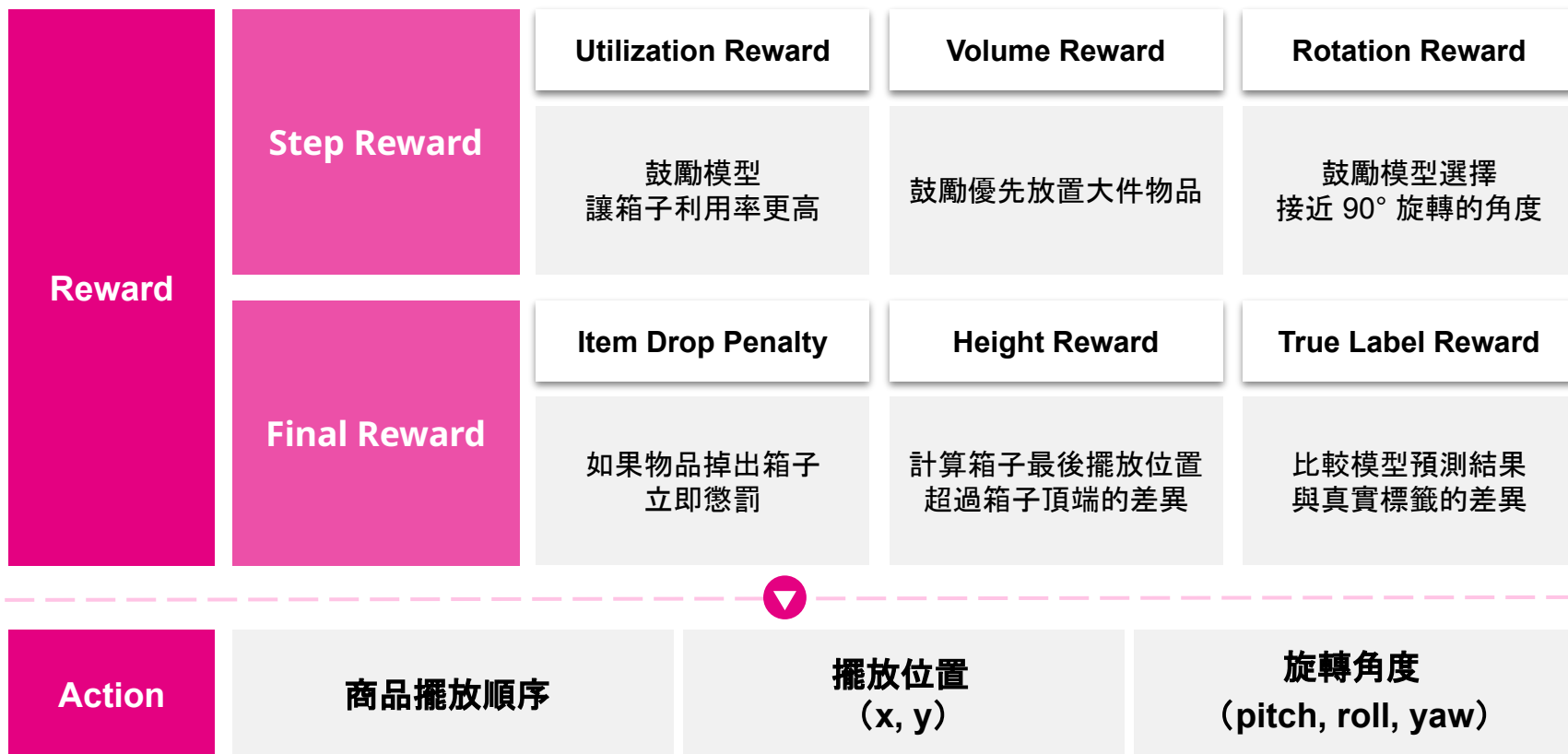
推薦最佳袋子模型參數

	整體準確率最高 (most accurate)				袋箱準確率均高 (most balanced)			
	偏重箱子				偏重袋子			
Max Volume Margin of Safety	1.5				1.4			
Min Volume Margin of Safety	1.5				1.0			
Max Size Margin of Safety	1				1.5			
Scale dict	PO-006	PO-007	PO-008	PO-009	PO-006	PO-007	PO-008	PO-009
	1.0	1.0	1.0	1.0	1.0	1.5	1.1	1.2

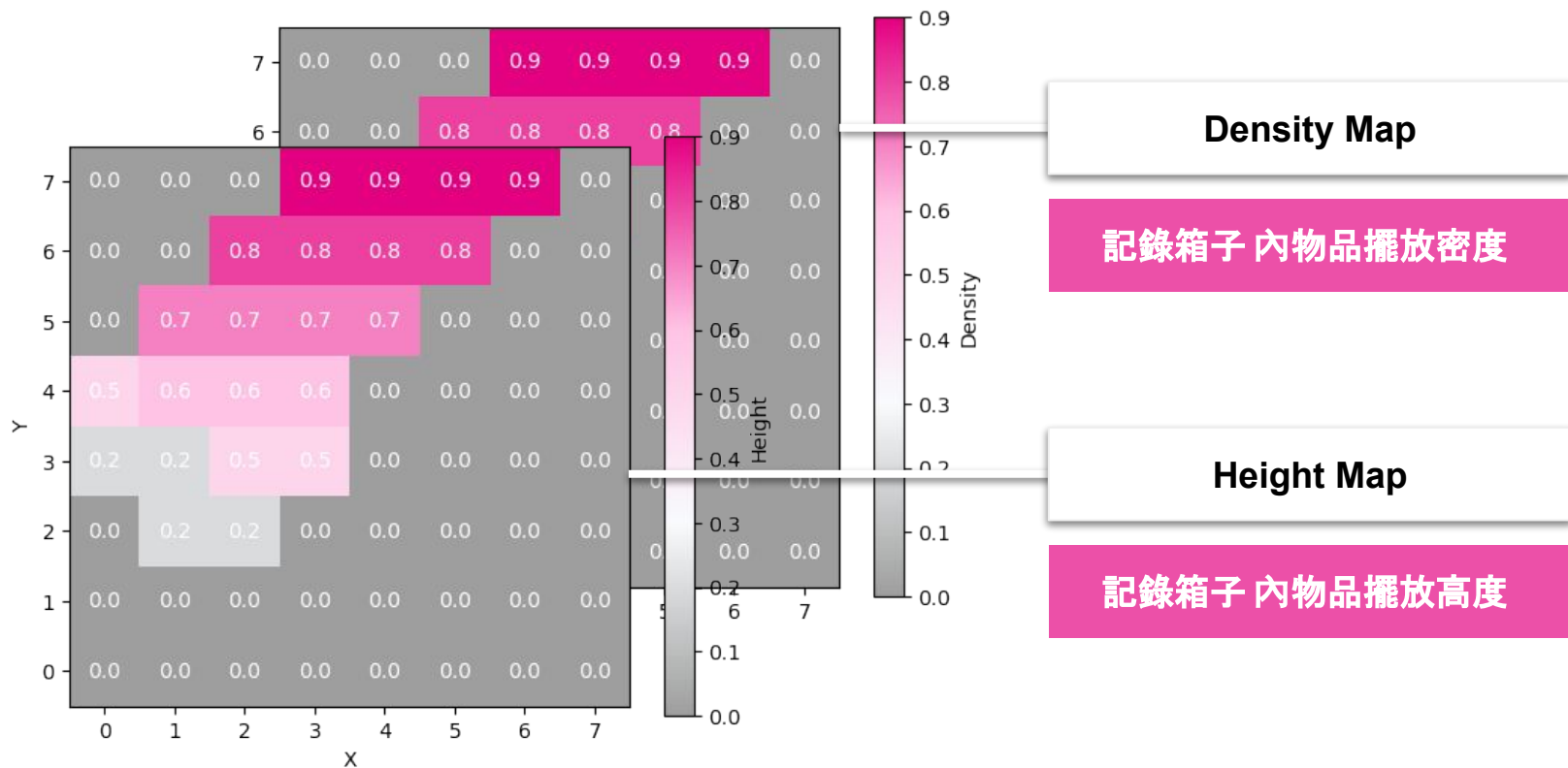
裝箱預測模型 | RandomForest 與 GradientBoosting 比較

	RandomForest	GradientBoosting
資料前處理	資料過濾	• 訂單資料: 選擇為高手推薦且裝箱時有採用的訂單當作訓練集。 • 包材資料: 過濾掉 BO/PO 系列袋子。
	特徵工程	• 計算商品的體積以及每筆訂單的總體積與總重量。 • 創建商品密度, 幫助模型區分不同商品特徵。 • 計算長寬高比, 改善裝箱判斷的準確性。
	資料外洩防護	• 根據商品組合 檢查訓練集與測試集中的重複資料, 避免資料外洩。 • 將測試集中的資料去除與訓練集相同的商品組合, 確保資料不會受到訓練集的影響。
訓練過程	依據 API_method 切分訓練集與驗證集並驗證: • 訓練集: API_method == '高手' 且 Adopt_YN == 'Y' (有採用) • 驗證集: API_method == 'QC歷史最佳包材' or '常溫3d模擬' 且 Adopt_YN == 'Y' (有採用)	

強化學習 | 設計架構

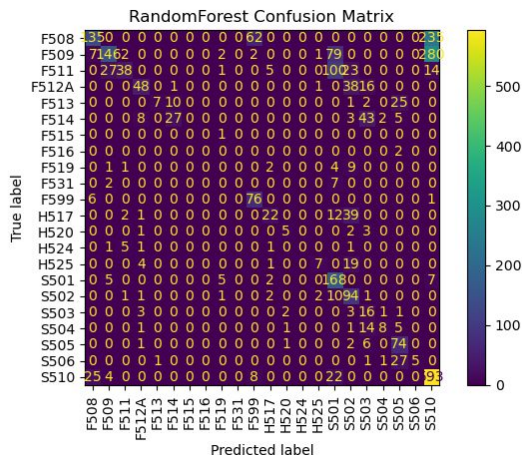


強化學習 | CNN Height Map + Density Map

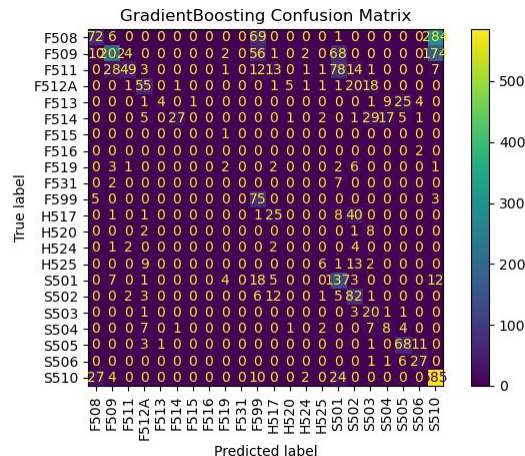


裝箱預測模型：RandomForest 與 GradientBoosting 比較

RandomForest



GradientBoosting



混淆矩陣

準確率

53.21%

裝載率

55.38%

52.30%

62.86%